МОСКОВСКИЙ ФИЗИКО-ТЕХНИЧЕСКИЙ ИНСТИТУТ
Физтех-школа радиотехники и компьютерных технологий

Отчёт по домашнему заданию №2
«Введение в архитектуру вычислительных систем»

Модуль ALU с регистром результата

Выполнила: Земляная Дарья Игоревна
Группа: Б01-404
Вариант: 2
zemlianaia.di@phystech.edu

23 марта 2026 г.

1 Документация модуля

1.1 Что делает этот модуль

Модуль `alu_register` — это арифметико-логическое устройство (АЛУ), которое умеет выполнять разные операции над двумя числами. Главная особенность: результат не появляется сразу на выходе, а сначала сохраняется в регистр и становится доступным только в следующем такте. Это нужно, чтобы результат был стабильным целый такт — так удобнее встраивать модуль в большие системы.

1.2 Как устроен модуль

Внутри модуль состоит из двух частей:

- **АЛУ** — часть, которая прямо сейчас вычисляет результат по входам (комбинационная логика)
- **Регистр** — ячейка памяти, которая запоминает результат по команде от тактового сигнала

Структурная схема показана на рисунке 1.

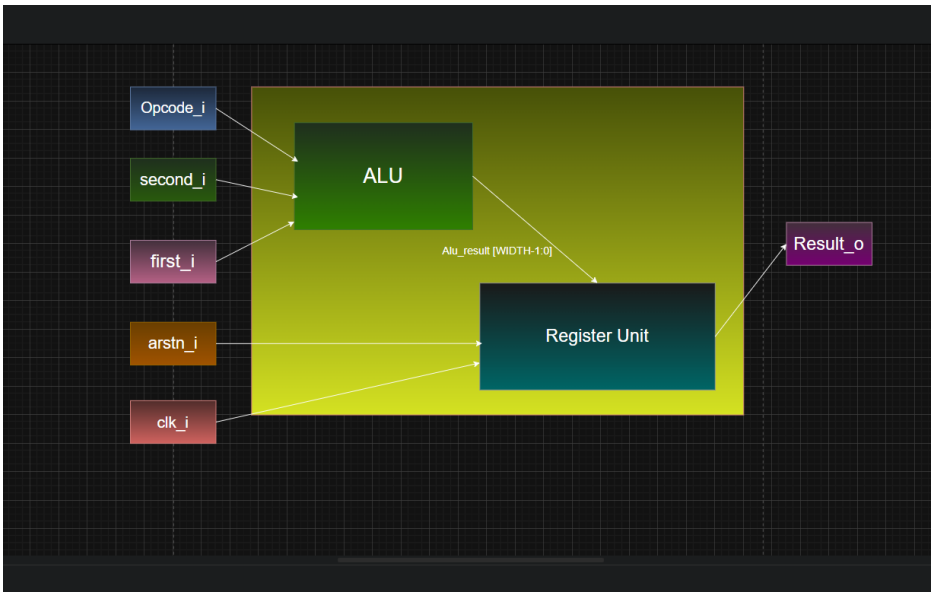

Рис. 1: Структурная схема модуля alu_register

1.3 Настройки

Модуль можно настроить под разную разрядность чисел:

Параметр	Что делает
WIDTH	Ширина операндов в битах (по умолчанию 8)

1.4 Входы и выходы

Сигнал	Ширина	Направление	Что это
clk_i	1	Вход	Тактовый сигнал — по его "тиканию"регистр запоминает результат
arstn_i	1	Вход	Сброс: если 0 — регистр сразу обнуляется (асинхронный, активный низкий
first_i	WIDTH	Вход	Первое число
second_i	WIDTH	Вход	Второе число
opcode_i	3	Вход	Код операции (какую операцию делать)
result_o	WIDTH	Выход	Результат (то, что лежит в регистре)

1.5 Как работает тактирование и сброс

Тактовый сигнал (clk_i) работает как метроном. По каждому положительному фронту (когда сигнал переключается с 0 на 1) регистр запоминает текущее значение с выхода АЛУ. Входные данные могут меняться когда угодно, но запомнятся только в момент такта.

Сброс (arstn_i) — это кнопка "очистить всё". Если подать на него 0, регистр обнуляется мгновенно, не дожидаясь такта. Такое поведение называется асинхронным сбросом.

1.6 Какие операции умеет выполнять

В зависимости от кода операции (opcode_i) модуль делает одно из действий:

Код	Операция	Что делает
000	NOR	Побитовое "НЕ-ИЛИ": ~(first | second)
001	AND	Побитовое "И": first & second
010	Signed ADD	Сложение со знаком (учитывает отрицательные числа)
011	Unsigned ADD	Обычное сложение
100	NOT second	Инвертирует второе число: ~second
101	XNOR	Побитовое "НЕ-Исключающее ИЛИ": ~(first ^ second)
110	Compare	Сравнивает числа: 1 если равны, иначе 0
111	Shift right	Логический сдвиг первого числа вправо на second бит

1.7 Как тестировали

Чтобы проверить, что всё работает, я написала тестовую программу (testbench), которая:

1. Генерирует тактовый сигнал
2. Подаёт на вход разные числа
3. Последовательно перебирает все коды операций
4. Записывает результаты в файл для просмотра временных диаграмм

Были проверены все 8 операций. Вот какие значения подавались:

Тест	first	second	Что ожидалось
NOR	AA (10101010)	55 (01010101)	00
AND	AA	55	00
Signed ADD	02	FE (это -2)	00
Unsigned ADD	02	03	05
NOT second	00	55	AA
XNOR	CC (11001100)	AA (10101010)	99 (10011001)
Compare	AA	AA	01
Shift right	80 (128)	03	10 (16)

2 Временные диаграммы

На рисунке 2 показано, как меняются сигналы во времени. Это результат работы симулятора GTKWave.

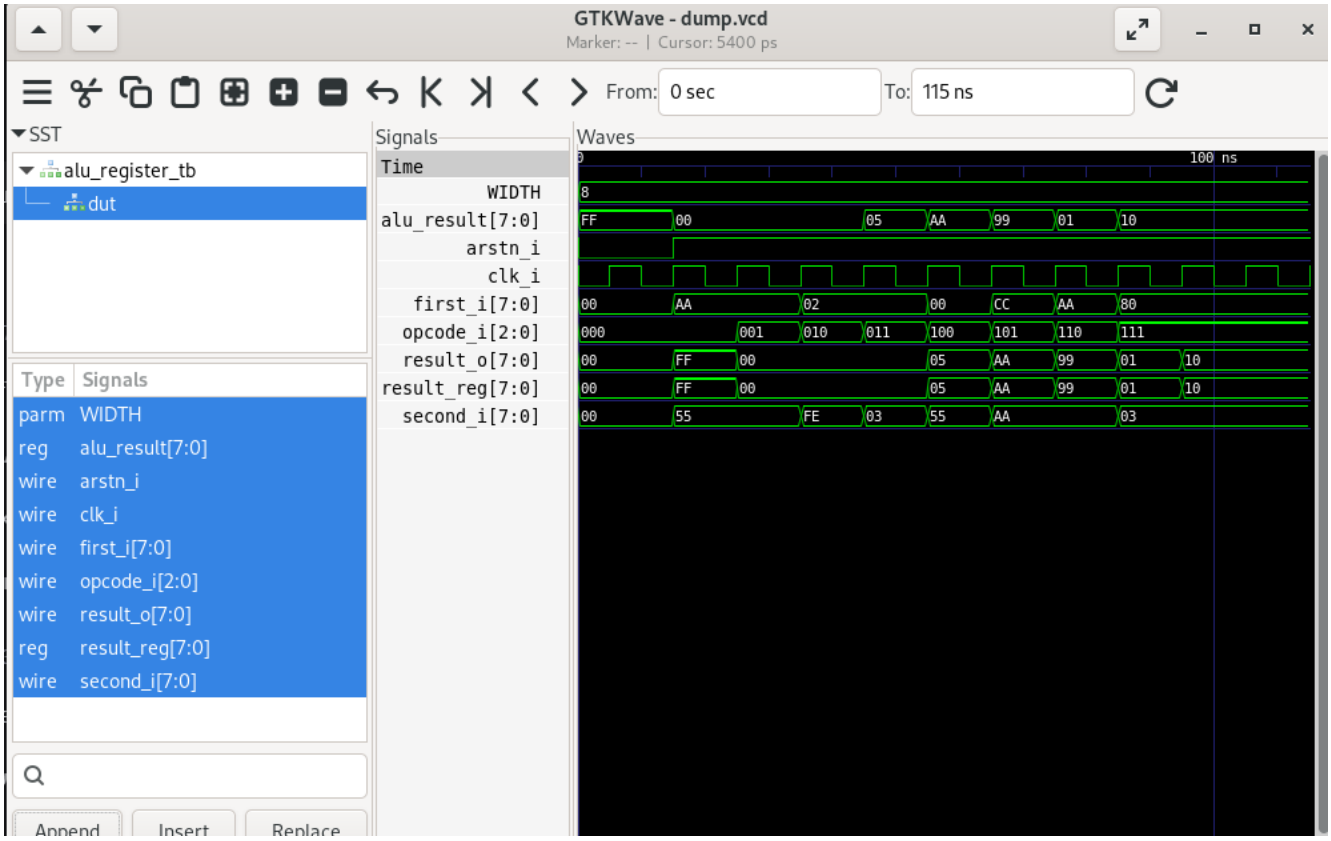

Рис. 2: Временные диаграммы работы модуля

Что видно на графиках:

- **clk** — тактовый сигнал: каждые 5 нс переключается (период 10 нс)
- **arstn** — сначала 0 (сброс), потом становится 1 и больше не меняется
- **first и second** — меняются каждые 10 нс, задавая новые числа для тестов
- **opcode** — код операции меняется от 0 до 7, по одному на тест
- **result** — результат появляется на один такт позже (это нормально, так работает регистр)

2.1 Результаты тестирования

Сравним ожидания с тем, что получилось на самом деле:

Тест	Операция	Ожидалось	Получилось	Вердикт
1	NOR	00	00	
2	AND	00	00	
3	Signed ADD	00	00	
4	Unsigned ADD	05	05	
5	NOT second	AA	AA	
6	XNOR	99	99	
7	Compare	01	01	
8	Shift right	10	10	

Все тесты пройдены успешно. Модуль работает корректно.

3 Заключение

В ходе выполнения домашнего задания:

- Разработан модуль ALU с регистром на Verilog
- Модуль поддерживает 8 арифметических и логических операций
- Написан testbench для проверки всех операций
- Проанализированы временные диаграммы в GTKWave

Все операции работают правильно, результаты совпадают с ожидаемыми. Модуль готов к использованию.